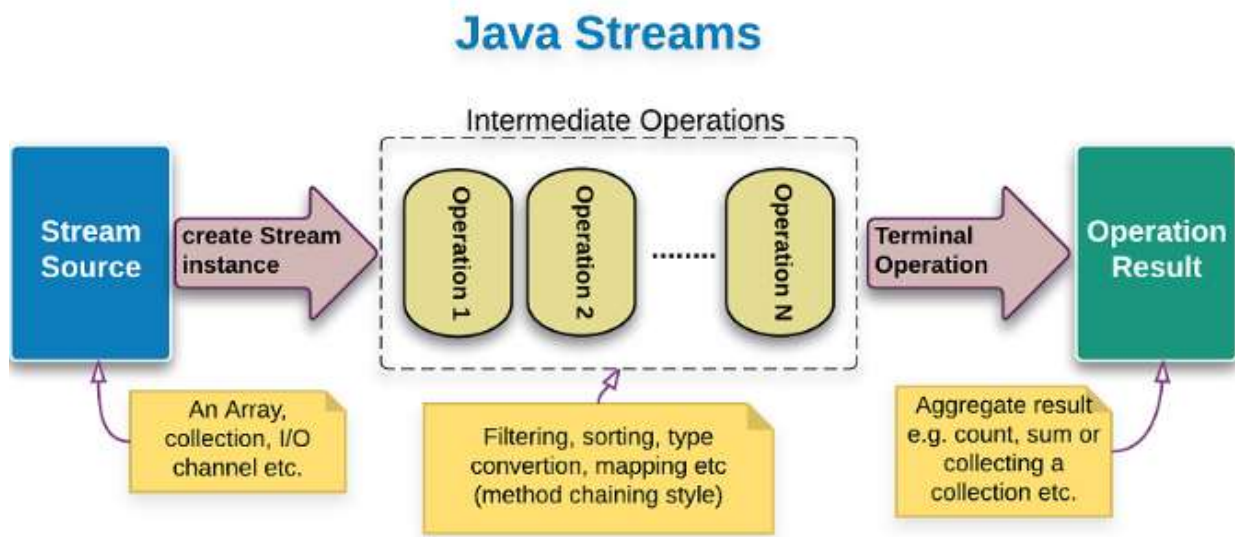


Java Streams

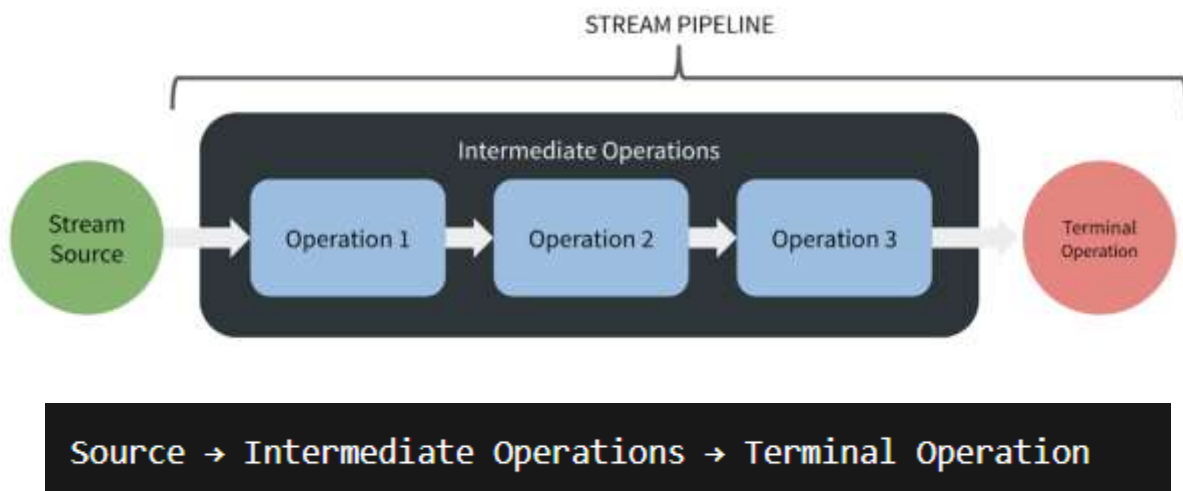
What is Stream?

Java Streams represent a **pipeline of operations** that process data from a source (List, Set, Map, Arrays).



- Do not store data
- Do not modify original collection
- Are lazily evaluated
- Can be consumed only once
- Support parallel processing

Stream Pipeline Structure



Types of Operations

Intermediate Operations (Lazy)

Return Stream:

- `filter()`
- `map()`
- `distinct()`
- `sorted()`
- `skip()`
- `limit()`

Terminal Operations (Trigger execution)

Return final result:

- `collect()`
- `reduce()`
- `count()`
- `min()`
- `max()`
- `findFirst()`
- `findAny()`
- `anyMatch()`
- `allMatch()`
- `noneMatch()`
- `average()`
- `sum()`

Why do we need Streams?

Before Java 8:

- External iteration (for-loops)
- More boilerplate
- Hard to parallelize
- Imperative style

With Streams:

- Declarative programming
- Clean transformation pipeline

- Functional style
- Built-in aggregation and grouping
- Easier parallel execution

Can we consume Stream only once?

Once terminal operation executes:

- Stream is closed
- Cannot reuse it
- Need to create new stream

Ex:

```
nums.stream().sorted().map(n -> n*2)
```

That is why → even though this line creating 3 streams will not consume much of the data coz once the stream is consumed that's all that is deleted.

Even though 3 operations exist,

Actual execution happens only once during terminal operation.

STREAM CREATION METHODS

```
Stream<String> stream = names.stream();  
Stream<String> stream2 = Arrays.stream(arr);  
Stream<Integer> intStream = Stream.of(1,2,3);  
Stream<Double> random = Stream.generate(Math::random).limit(5);
```

STREAMS ON PRIMITIVE & STRING COLLECTIONS

```
List<Integer> arr = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
```

FILTER (Intermediate Operation)

Even Numbers & Odd Numbers → Filter

```
List<Integer> even = arr.stream()
    .filter(n -> n % 2 == 0)
    .toList();

List<Integer> odd = arr.stream()
    .filter(n -> n % 2 != 0)
    .collect(Collectors.toList());
```

Square of Even Numbers > 5 → Filter + Map

```
List<Integer> sqevn5 = arr.stream()
    .filter(n -> n > 5)
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .toList();
```

REDUCE (Terminal Operation)

Summing All Numbers

```
int sum = arr.stream()
    .reduce(0, (a, b) -> a + b);

int sumNoIdentity = arr.stream()
    .reduce(0, Integer::sum);

Optional<Integer> sumopt = arr.stream()
    .reduce((a, b) -> a + b);
```

Using mapToInt()

```
int summ = arr.stream()
    .mapToInt(a -> a)
    .sum();
```

MAX / MIN

```
int maxx = arr.stream()
    .max(Integer::compareTo)
    .get();

int minn = arr.stream()
    .min(Integer::compareTo)
    .get();
```

Safer version:

```
List<Integer> arr2 = Arrays.asList();

int min2 = arr2.stream()
    .min(Integer::compareTo)
    .orElse(-1);
```

Using reduce for max/min

```
Integer maxx = arr.stream()
    .reduce(Integer.MIN_VALUE, Integer::max);

Optional<Integer> maxxOpt = arr.stream()
    .reduce(Integer::max);

Integer maxxOrElse = arr.stream()
    .reduce(Integer::max)
    .orElse(Integer.MIN_VALUE);
```

distinct()

Removes duplicate elements.

```
List<Integer> arr3 = Arrays.asList(1,1,2,3,5,6,7);

List<Integer> dis = arr3.stream()
    .distinct()
    .toList();
```

sorted()

Ascending

```
List<Integer> arr4 = Arrays.asList(1,4,3,2,5,7,8,6,9);

List<Integer> sortAsc = arr4.stream()
    .sorted()
    .toList();
```

Descending

```
List<Integer> sortDesc = arr4.stream()
    .sorted(Comparator.reverseOrder())
    .toList();
```

Combined Operations

Sum of even numbers

```
int sumEven = arr.stream()
    .filter(n -> n % 2 == 0)
    .reduce(0, Integer::sum);
```

Sum of odd numbers

```
int sumOdd = arr.stream()
    .filter(n -> n % 2 != 0)
    .reduce(0, Integer::sum);
```

Sum of Squares of Even Numbers

```
int sumEvnSq = arr.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(0, Integer::sum);
```

Safer Optional version:

```
List<Integer> arr5 = Arrays.asList();

Optional<Integer> result = arr5.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(Integer::sum);

int res = arr5.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(Integer::sum)
    .orElse(-1);
```

Product Using reduce()

```
int prod = arr.stream()
    .reduce(1, (a, b) -> a * b);

prod = arr.stream()
    .reduce((a, b) -> a * b)
    .orElse(-1);
```

findFirst()

First Element greater than 7, greater than 1000

```
int eleGt7 = arr.stream()
    .filter(n -> n > 7)
    .findFirst()
    .get();

int eleGt1000 = arr.stream()
    .filter(n -> n > 1000)
    .findFirst()
    .orElse(-1);

Optional<Integer> eleGt1000Opt =
    arr.stream()
        .filter(n -> n > 1000)
        .findFirst();
```

anyMatch / allMatch / noneMatch

Any element (>10), All are even, none of them are negative

```
List<Integer> nums = Arrays.asList(3,5,7,12,4);

boolean result_check = nums.stream()
    .anyMatch(n -> n > 10);

boolean even_check = nums.stream()
    .allMatch(n -> n % 2 == 0);

boolean negative_check = nums.stream()
    .noneMatch(n -> n < 0);
```

count()

```
long count = nums.stream()
    .filter(n -> n > 5)
    .count();
```

Convert List to Map → String and it's Length

```
List<String> names = Arrays.asList("Surya","Mahesh","Kiran");

Map<String,Integer> map = names.stream()
    .collect(Collectors.toMap(
        name -> name,
        name -> name.length()
    ));
```

Advanced Reduce Practice

Product

```
int product = nums.stream()
    .reduce(1, (a, b) -> a * b);
```

Sum of Squares

```
int sumOfSquares = nums.stream()
    .map(n -> n * n)
    .reduce(0, Integer::sum);
```


Longest String

```
Optional<String> longest_string =  
    names.stream()  
        .reduce((a, b) ->  
            a.length() > b.length() ? a : b);
```

Advanced Sorting & Finding

```
List<Integer> numbers = Arrays.asList(  
    1,1,1,1,5,7,73,2,240,111,2,3,7,6,-1,100,45,120,240,111);
```

Least

```
int least = numbers.stream()  
    .sorted()  
    .findFirst()  
    .orElse(-1);
```

Second Highest

```
int second_highest = numbers.stream()  
    .sorted(Comparator.reverseOrder())  
    .skip(1)  
    .findFirst()  
    .orElse(-1);
```

Distinct + Count + Sort

Distinct Numbers list & Count of Distinct Numbers

```
List<Integer> distinct_numbers =  
    numbers.stream()  
        .distinct()  
        .toList();  
  
long distinct_num_count =  
    numbers.stream()  
        .distinct()  
        .count();
```

Sort all distinct elements in list

```
List<Integer> distinct_sort =  
    numbers.stream()  
        .distinct()  
        .sorted()  
        .toList();
```

String Filtering

```
String namewiths = names.stream()  
    .filter(s -> s.startsWith("s"))  
    .findFirst()  
    .orElse("");  
  
String nameendswithn = names.stream()  
    .filter(s -> s.endsWith("n"))  
    .findFirst()  
    .orElse("");
```

PART 2 – ClassStreams (Employee Example)

```
class Employee {  
    private int id;  
    private String name;  
    private int age;  
    private String department;  
    private double salary;  
  
    public Employee(int id, String name,  
                    int age, String department,  
                    double salary) {  
        this.id = id;  
        this.name = name;  
        this.age = age;  
        this.department = department;  
        this.salary = salary;  
    }  
}
```

```

    public int getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    public String toString() {
        return id+" - "+name+" - "+age+" - "+department+" - "+salary;
    }
}

```

```

List<Employee> employees = Arrays.asList(
    new Employee(1,"Surya",25,"IT",60000),
    new Employee(2,"Mahesh",30,"HR",50000),
    new Employee(3,"Kiran",28,"IT",75000),
    new Employee(4,"Anil",35,"Finance",90000),
    new Employee(5,"Suresh",40,"IT",120000),
    new Employee(6,"Ravi",22,"HR",40000),
    new Employee(7,"John",29,"Finance",85000),
    new Employee(8,"Sam",31,"IT",95000)
);

```

Filtering

List of Employees whose Salary > 70000

```

employees.stream()
    .filter(e -> e.getSalary() > 70000)
    .toList();

```

List of all IT Employees

```

employees.stream()
    .filter(e -> e.getDepartment().equals("IT"))
    .toList();

```

⚠ Always use .equals() not == for Strings.

Map

List of all employee names

```
employees.stream()
    .map(Employee::getName)
    .toList();
```

List of all employee salaries

```
employees.stream()
    .map(Employee::getSalary)
    .toList();
```

Sum of Salaries

```
double sum_salary = employees.stream()
    .map(Employee::getSalary)
    .reduce(0.0, Double::sum);
```

Max / Min

Employee with Highest salary

```
Employee highest_emp =
    employees.stream()
        .max(Comparator.comparing(Employee::getSalary))
        .orElse(null);
```

Employee with Least Age – Youngest Employee

```
Employee youngest_emp =
    employees.stream()
        .min(Comparator.comparing(Employee::getAge))
        .orElse(null);
```

Sorting

Sort Employees – by Salary ascending order

```
employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary))
    .toList();
```

Sort Employees – by Salary descending order

```
employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .toList();
```

Sort Employees – sort by dept then sort by Salary ascending order

```
employees.stream()
    .sorted(Comparator
        .comparing(Employee::getDepartment)
        .thenComparing(Employee::getSalary))
    .toList();
```

findFirst / skip

Find employee with Second highest salary

```
Employee secondHighest =
    employees.stream()
        .sorted(Comparator.comparing(Employee::getSalary).reversed())
        .skip(1)
        .findFirst()
        .orElse(null);
```

Match Operations

```
boolean anyHighSalary =
    employees.stream()
        .anyMatch(e -> e.getSalary() > 70000);

boolean all_emp_age_gt20 =
    employees.stream()
        .allMatch(e -> e.getAge() > 20);
```

```
boolean none_sales_emp =  
    employees.stream()  
        .noneMatch(e -> e.getDepartment().equals("Sales"));
```

Average Salary of Employees

```
double avgSalary =  
    employees.stream()  
        .mapToDouble(Employee::getSalary)  
        .average()  
        .orElse(0);
```

Grouping – Group by Dept & Count Number of Employees in each department

```
Map<String,List<Employee>> groupByDept =  
    employees.stream()  
        .collect(Collectors.groupingBy(Employee::getDepartment));  
  
Map<String,Long> deptCount =  
    employees.stream()  
        .collect(Collectors.groupingBy(  
            Employee::getDepartment,  
            Collectors.counting()  
        ));
```

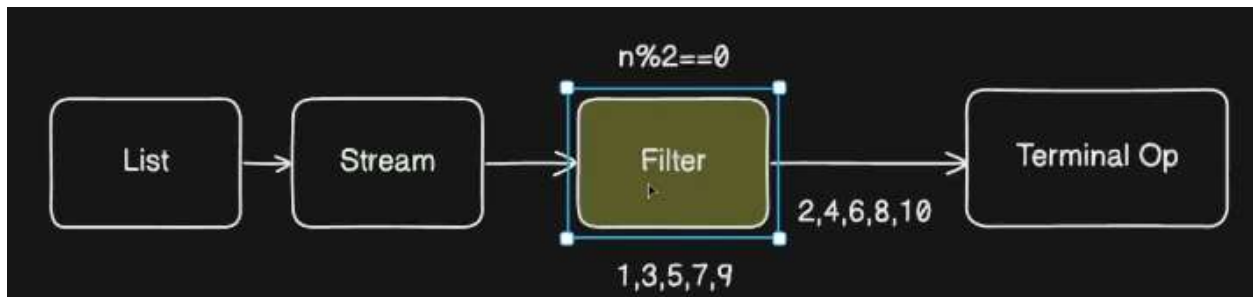
INTERVIEW QUESTIONS – LEVEL 1 (Super Easy)

```
List<Integer>array = Arrays.asList(1,1,1,2,3,4,5,6,7,8,9,10,100,1000);
```

1. Even Numbers – Filter

```
array.stream()  
    .filter(n -> n%2==0)  
    .forEach(System.out::println);
```

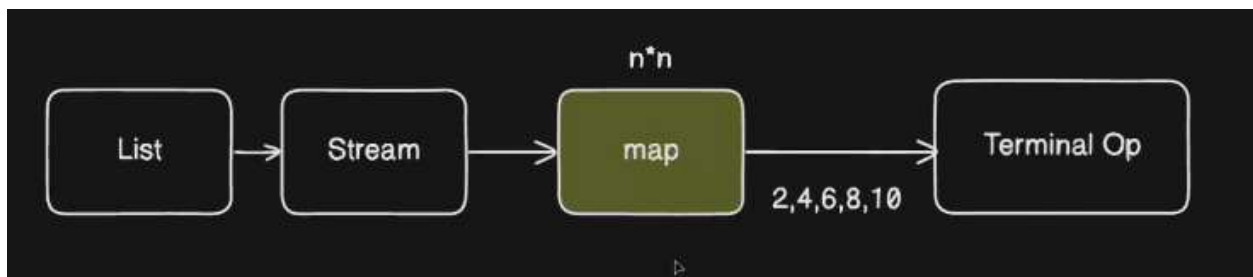
Internal Working:



2. Convert Numbers into List of Their Squares

```
array.stream()  
    .map(n -> n*n)  
    .forEach(System.out::println);
```

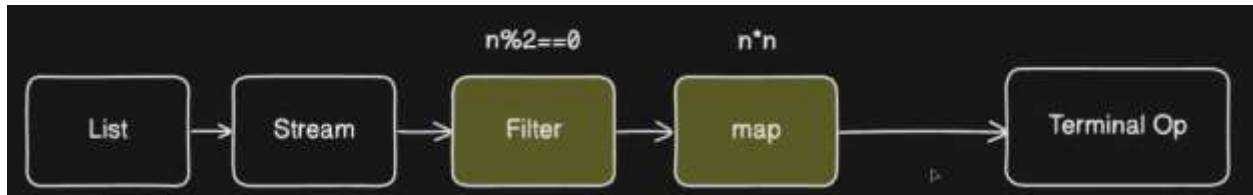
Internal working:



3. Square Even Numbers from List

```
array.stream()  
    .filter(n -> n%2==0)  
    .map(n -> n*n)  
    .forEach(System.out::println);
```

Internal Working:



4. Find First Number Greater Than 10

```
array.stream()  
  .filter(n -> n>10)  
  .findFirst();
```

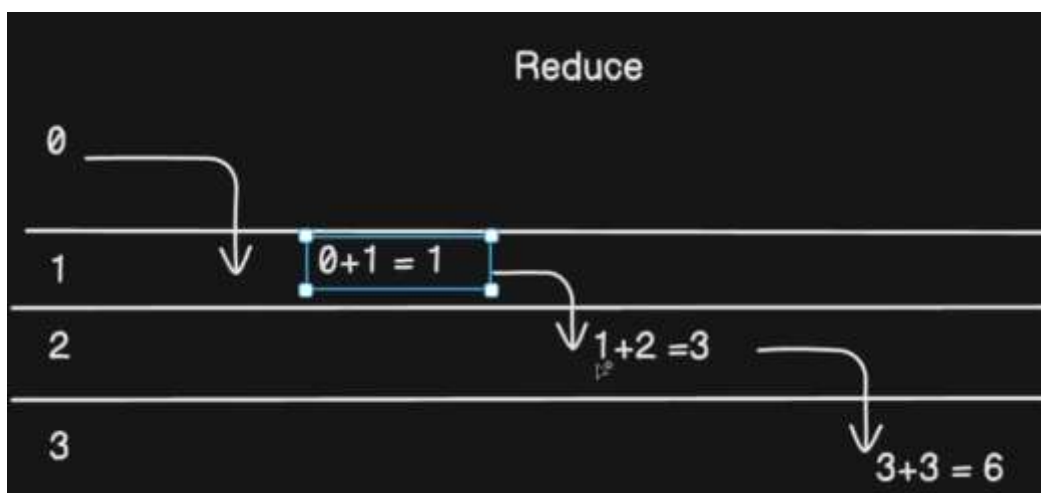
5.Count numbers are greater than 5

```
array.stream()  
  .filter(n -> n>5)  
  .count();
```

6.Find the sum/product of all numbers in list

```
array.stream().reduce(Integer::sum);  
array.stream().reduce(0,Integer::sum);  
array.stream().reduce(1,(a,b)->a*b);
```

Internal working:



7.Find sum of Even numbers only

```
array.stream()  
    .filter(n -> n%2==0)  
    .reduce(0,Integer::sum);
```

Flow

Stream → filter → reduce(accumulate)

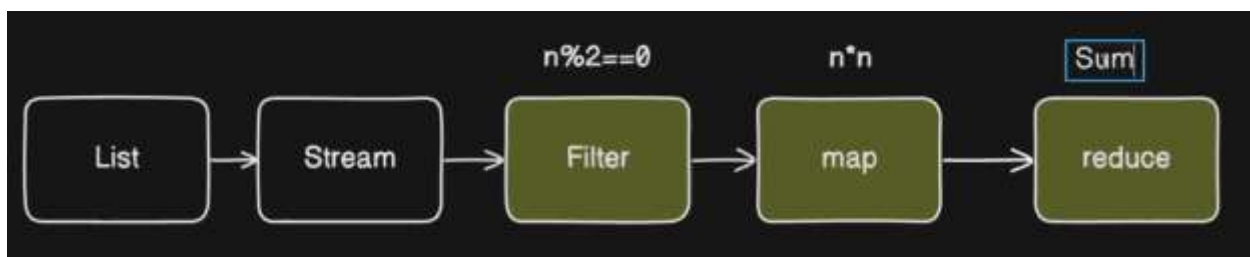
8.find maximum number in the list

```
array.stream().reduce(Integer::max);
```

9.Find sum squares even numbers in a list

```
array.stream()  
    .filter(n -> n%2==0)  
    .map(n -> n*n)  
    .reduce(Integer::sum);
```

Internal Working:



Find First Non Empty String in the list

```
sarr = Arrays.asList("", "Mahesh", "Arush", "Arya");  
System.out.println(sarr.stream().filter( String s -> !s.isEmpty()).findFirst());  
System.out.println(sarr.stream().filter( String s -> !s.isEmpty()).findFirst().get());
```

```
[Mahesh, Arush, Arya]  
Optional[Mahesh]  
Mahesh
```

Interview Questions – Level 2 Easy

10. Find all distinct elements from a list → Remove duplicates

```
array.stream().distinct().toList();
```

Flow:

```
Stream → distinct() → collect
```

11. Find Average of numbers in a list

```
array.stream().mapToInt(n -> n).average().orElse(-1);
```

→ Stream to be Converted into IntStream

Internal working:



12. Sort a list of integers in Asc and Desc order

```
array.stream().sorted();  
array.stream().sorted(Comparator.reverseOrder());
```

Flow:

```
Stream → sorted → terminal
```

```
List<String> sarr = Arrays.asList("Mahesh","Arush","Arya"))
```

13. Count Number of Strings Starts with a specific letter like 'A'

```
sarr.stream()  
    .filter(s -> s.startsWith("A"))  
    .count();
```

14. Join all strings in a list into a single comma separated string

15. Join all strings in a list into a single comma separated string, with prefix and suffix

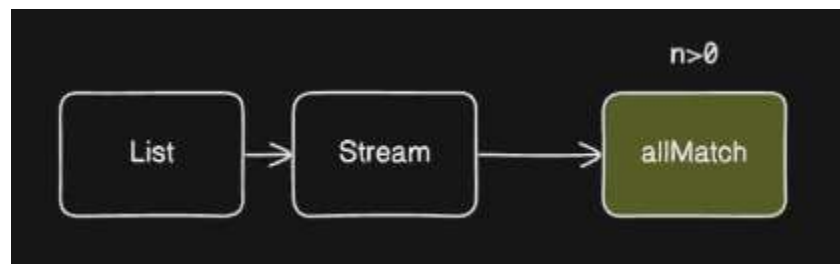
```
sarr.stream()
    .collect(Collectors.joining(","));

sarr.stream()
    .collect(Collectors.joining(",","[" ,"]"));
```

16. Check if all the elements are positive numbers → allMatch()

```
array.stream().allMatch(n -> n>0);
```

All match Internal working:



17. Check if Any number is Divisible by 3 → anyMatch()

```
array.stream().anyMatch(n -> n%3==0);
```

18. Flatten the list of List

```
listofLists.stream()
    .flatMap(List::stream)
    .toList();
```

Flow:

```
Stream<List>
    → flatMap
    → Stream<Integer>
    → collect
```

Interview Questions – Level 3 Intermediate

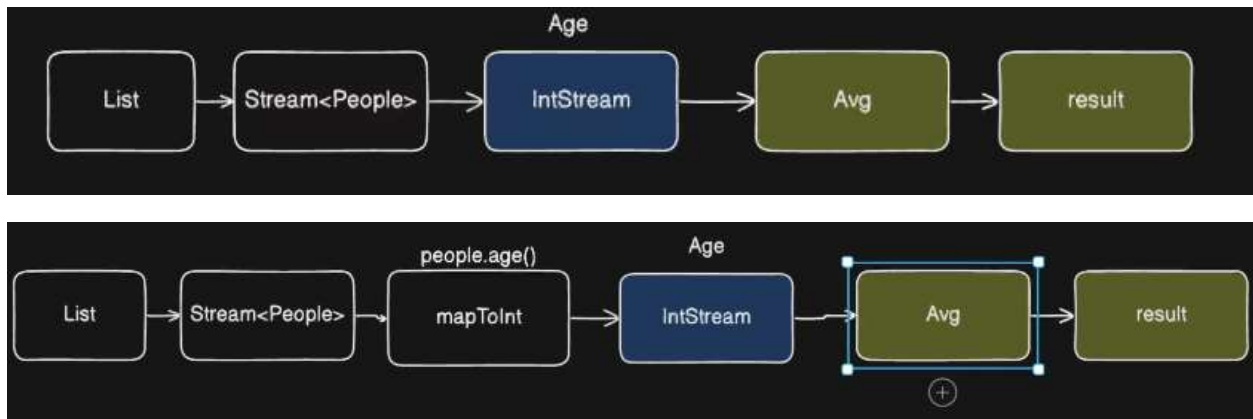
19. Sort Employees By Salary

```
employees.stream()  
    .sorted(Comparator.comparing(Employee::getSalary))  
    .toList();
```

20. Calculate the average age of list of Employees Objects using Java Streams

```
employees.stream()  
    .mapToInt(Employee::getAge)  
    .average();
```

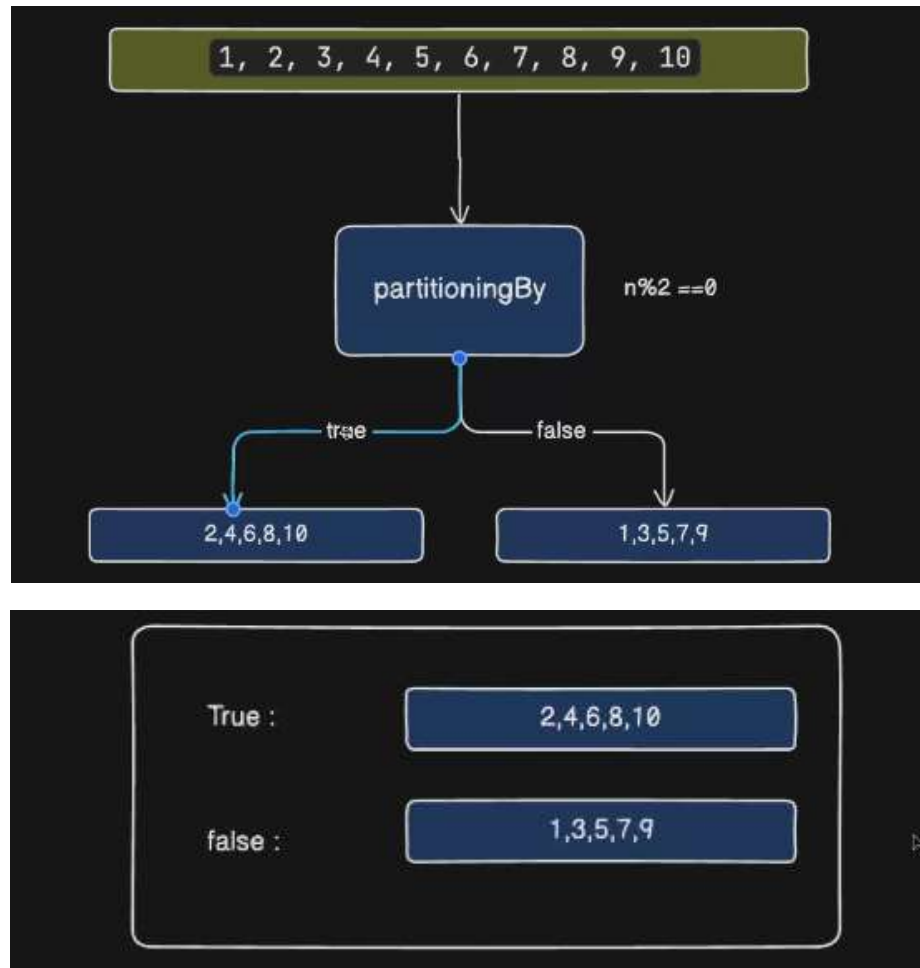
Internal Working:



21. Partition numbers into Even and Odd

```
nums.stream()  
    .collect(Collectors.partitioningBy(n -> n%2==0));
```

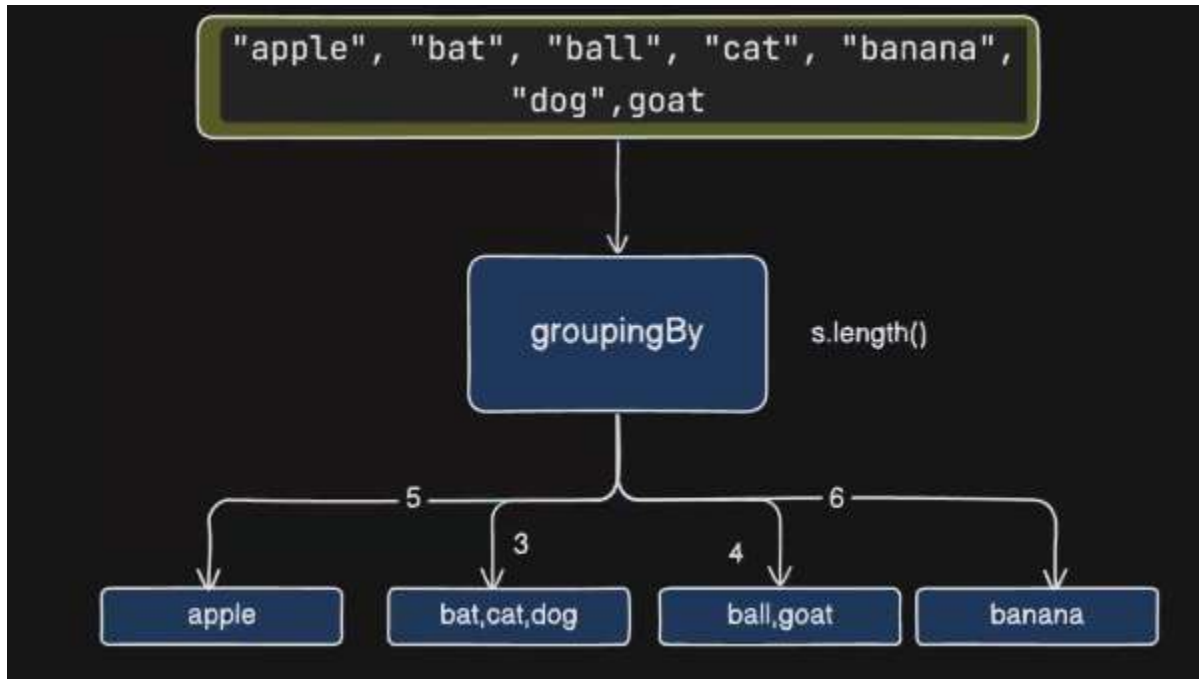
Internal Working:



22. Group a list of words by their length using streams

```
words.stream()  
    .collect(Collectors.groupingBy(String::length));
```

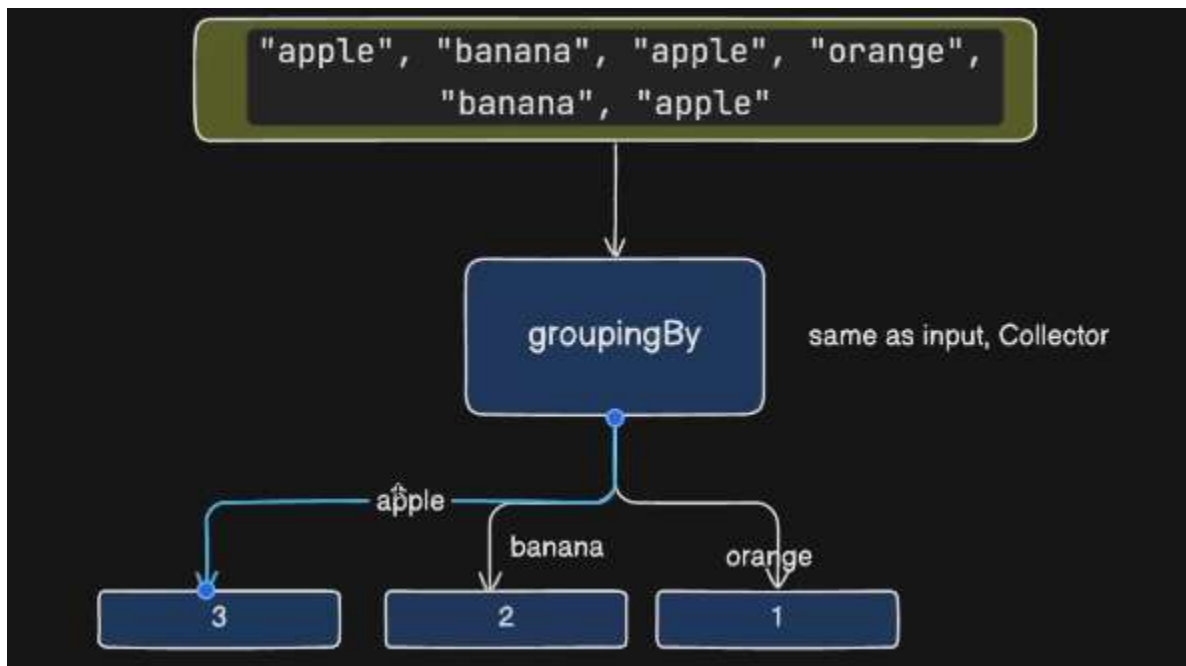
Internal working:



23. Count Occurrences of Each Element in a list

```
words.stream()  
    .collect(Collectors.groupingBy(  
        Function.identity(),  
        Collectors.counting()  
    ));
```

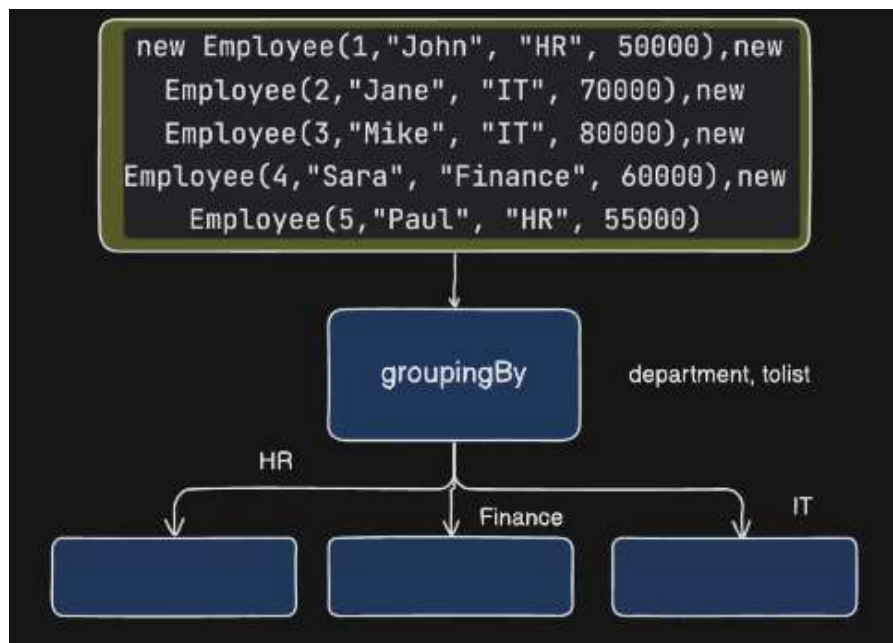
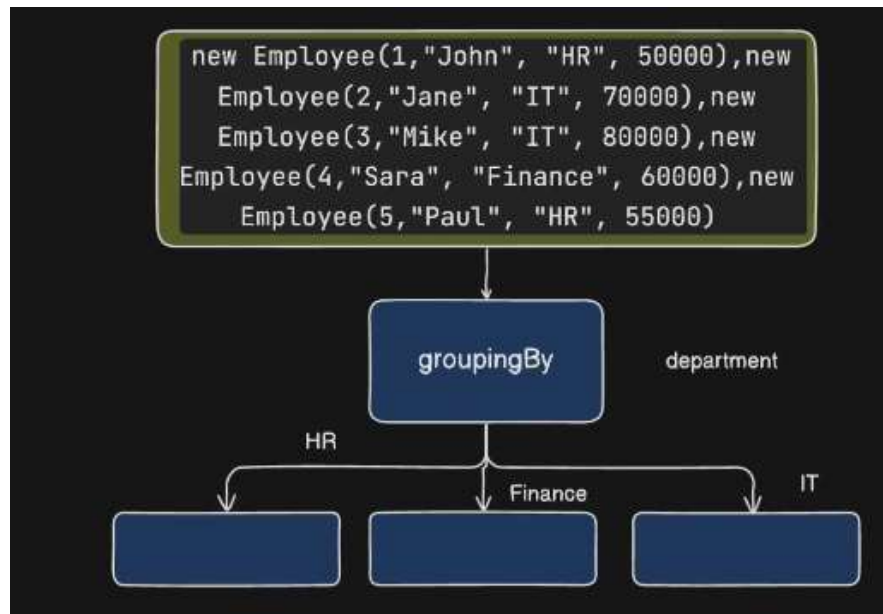
Internal Working:



25. Group Employees by Department

```
employees.stream()  
    .collect(Collectors.groupingBy(Employee::getDepartment));
```

Internal Working:



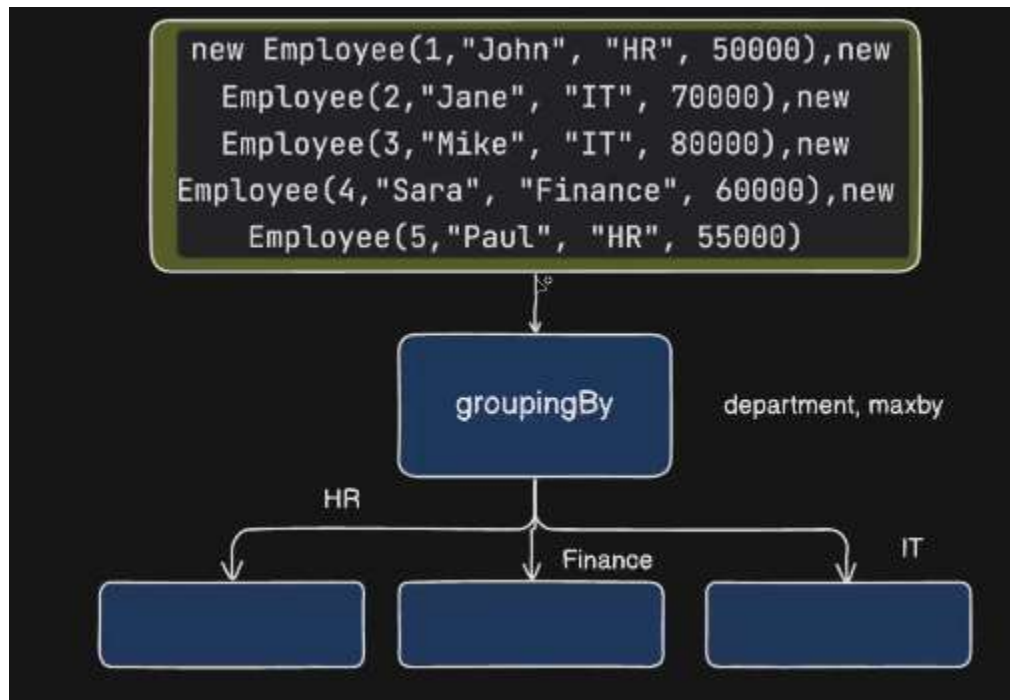
26.Group employees by department and calculate average salary

```
employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));
```

27.Highest Paid Employee in each department

```
employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(
            Comparator.comparing(Employee::getSalary)
        )
    ));
```

Internal Working:



28. Find Department wise Employee Count

```
System.out.println("Find department wise employee count");

Map<String, Long> deptWiseEmployeeCount = employees.stream()
                                                    .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()));

System.out.println(deptWiseEmployeeCount);
```

29. Find all departments with more than 2 Employees

```
System.out.println("Find all departments with more than 2 Employees");
System.out.println(deptWiseEmployeeCount.entrySet().stream().filter(Entry<String, Long> e -> e.getValue()>2).toList());
System.out.println(deptWiseEmployeeCount.entrySet().stream().filter(Entry<String, Long> e -> e.getValue()>2).map(Entry<String, Long> e -> e.getKey()).toList());
System.out.println(deptWiseEmployeeCount.entrySet().stream().filter(Entry<String, Long> e -> e.getValue()>2).map(Map.Entry::getKey).toList());
```

30. Find all departments with more than 2 Employees → Using Single Stream

```
System.out.println("Find all departments with more than 2 Employees into Single Stream");
employees.stream() Stream<Employee>
    .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting())) Map<String, Long>
    .entrySet() Set<Entry<String, Long>>
    .stream() Stream<Entry<String, Long>>
    .filter(Entry<String, Long> e -> e.getValue()>2)
    .map(Entry<String, Long> e -> e.getKey()) Stream<String>
    .toList() List<String>
    .forEach(String d -> System.out.println(d));
```

31. Find Department wise Average Salary

```
System.out.println("Department wise Average Salary");
System.out.println(
    {
        employees.stream()
            .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.averagingDouble(Employee::getSalary)))
    }
);
```

32. Find Department with Highest Average Salary

```
employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)))
    .entrySet().stream()
    .sorted(Comparator.comparing(
        Map.Entry::getValue,
        Comparator.reverseOrder()))
    .findFirst()
    .get()
    .getKey();
```

Flow:

```
Stream → groupingBy → entrySet  
        → sort(desc by value)  
        → findFirst  
        → getKey
```

33.Find Department with Second Highest Average Salary

```
System.out.println("Find Department with Second Highest Average Salary");  
System.out.println(  
    employees.stream().stream()<Employee>  
        .collect(Collectors.groupingBy(Employee::getDepartment,Collectors.averagingDouble(Employee::getSalary)))<Map<String, Double>  
        .entrySet()><Set<Entry<String, Double>>  
        .stream()><Stream<Entry<String, Double>>  
        .sorted(Comparator.comparing(Entry<String, Double> e -> e.getValue(),Comparator.reverseOrder()))  
        .skip(1).findFirst()<Optional<Entry<String, Double>>  
        .get().getKey());
```

34.Find Employee with Second Highest Salary

```
System.out.println("Find Employee with Second Highest Salary");  
System.out.println(employees.stream().sorted(Comparator.comparing(Employee::getSalary)).skip(1).findFirst().get());
```

35.Find Frequency of each character in String

```
word.chars()  
    .mapToObj(c -> (char)c)  
    .collect(Collectors.groupingBy(  
        Function.identity(),  
        Collectors.counting()  
    ));
```

36.Find Most Frequent character in String

```
System.out.println("Find Most Frequent character in String");  
System.out.println(  
    word.chars().<IntStream>  
        .mapToObj((int c) -> (char)c)<Stream<Character>  
        .collect(Collectors.groupingBy(Function.identity(),Collectors.counting()))<Map<Character, Long>  
        .entrySet()><Set<Entry<Character, Long>>  
        .stream()><Stream<Entry<Character, Long>>  
        .collect(Collectors.maxBy(Comparator.comparing(Entry<Character, Long> e -> e.getValue())))<Optional<Entry<Character, Long>  
        .get()><Entry<Character, Long>  
        .getKey()  
    );
```

```

word.chars()
    .mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(
        Function.identity(),
        Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .get()
    .getKey();

```

37. Find First Non Repeating Character

```

String str = "swiss";
System.out.println("String is : "+str);
System.out.println("Find First Non Repeating Character");
System.out.println(
    str.chars().intStream()
        .mapToObj( int c -> (char)c) Stream<Character>
        .collect(Collectors.groupingBy(Function.identity(),Collectors.counting())) Map<Character, Long>
        .entrySet() Set<Entry<...>>
        .stream() Stream<Entry<...>>
        .filter( Entry<Character, Long> e -> e.getValue()==1)
        .findFirst() Optional<Entry<...>>
        .get() Entry<Character, Long>
        .getKey()
);

```

```

str.chars()
    .mapToObj(c -> (char)c)
    .collect(Collectors.groupingBy(
        Function.identity(),
        Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue()==1)
    .findFirst()
    .get()
    .getKey();

```

Interview Questions – Level 4 Hard

38. Frequency of Employee Names with First Letter

```
employees.stream()
    .map(Employee::getName)
    .collect(Collectors.groupingBy(
        e -> e.charAt(0),
        Collectors.counting()
    ));
```

```
System.out.println("Frequency of Employee Names with First Letter");
System.out.println(
    employees.stream()
        .map(Employee e -> e.getName())
        .collect(Collectors.groupingBy(
            String e -> e.charAt(0), Collectors.counting()
        )));
```

39. Find the most common first letter among all employee names

```
employees.stream()
    .map(Employee::getName)
    .collect(Collectors.groupingBy(
        e -> e.charAt(0),
        Collectors.counting()))
    .entrySet().stream()
    .sorted(Comparator.comparing(
        Map.Entry::getValue,
        Comparator.reverseOrder()))
    .findFirst()
    .get()
    .getKey();
```

```
Stream → map(name)
        → groupingBy(first letter)
        → entrySet
        → sort(desc by count)
        → findFirst
```

